

Similarity Index of Cellular Automata

Erick Jesse Angeles López¹[0009–0001–0218–2341] and Sukanta Das²

¹ Instituto Politécnico Nacional, Escuela Superior de Cómputo, ALIROB Laboratory, Ciudad de México, México

angeleslopezerickjesse@gmail.com

² Indian Institute of Engineering Science and Technology, Shibpur, Howrah, West Bengal, India

Abstract. TODO: abstract.

Keywords: Cellular Automata · Similarity Index · Spectral Moments · Transition Graph · Game of Life.

1 Introduction

Los autómatas celulares son sistemas complejos, funciones que definen comportamientos muchas veces caóticos pero otras veces fácilmente predecibles. Múltiples investigaciones han realizado clasificaciones para determinar comportamientos diferentes, como la clasificación de Wolfram, de TODO o el conjunto de reglas life-like. Estas clasificaciones suelen diseñarse según su comportamiento, pero ¿Qué tan parecidas son estas reglas? y ¿Que tan diferentes son de otro conjunto de reglas?

Actualmente las clasificaciones es la única medida que podemos utilizar para definir si dos reglas son parecidas, o directamente comparando su comportamiento exacto. Sin embargo, no existe una medida que nos permita comparar que tan similares son. El propósito de esta investigación es definir un método que nos permita comparar el comportamiento de cualesquiera dos reglas y medir su grado de similitud.

Para conocer las similitudes es necesario identificar que relaciones exactas ya existen entre diferentes dominios, encontrar equivalencia y usarlas como métrica de similitud para los algoritmos propuestos, que esencialmente se reducen al cálculo de momentos espectrales con un método que identifica el incremento de la masa del atractor.

2 Preliminaries

Los autómatas celulares son una colección de células agrupadas en una red, donde cada una de ellas adquiere un estado de un conjunto finito. El sistema evoluciona en pasos de tiempo discretos y, en cada paso, todas las células actualizan su estado de manera síncrona en función del estado de sus células vecinas [5,6,4].

Definition 1. *Un autómata celular es una 4-tupla $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ donde:*

- $\mathcal{L} \subseteq \mathbb{Z}^D$ es el espacio celular D -dimensional. Cada célula se identifica con un vector $\vec{i} \in \mathcal{L}$ que representa su posición en el espacio.

This work is done as a project in the Indian Summer School on Cellular Automata 2026.

- $\mathcal{S} = \{0, 1, \dots, k-1\}$ es el conjunto finito de k estados. El estado de la célula $\vec{i}$ en el tiempo t se denota por $\sigma_{\vec{i}}(t) \in \mathcal{S}$.
- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \dots, \vec{v}_m)$ es la tupla ordenada de los m vectores de desplazamiento que definen la vecindad. Los vecinos de la célula $\vec{i}$ se encuentran en las posiciones $\vec{i} + \vec{v}_j$, con $j = 1, \dots, m$.
- $\mathcal{R} : \mathcal{S}^m \rightarrow \mathcal{S}$ es la regla local, que asigna el nuevo estado de una célula a partir de los estados de sus m vecinos:

$$\sigma_{\vec{i}}(t+1) = \mathcal{R} \left(\sigma_{\vec{i}+\vec{v}_1}(t), \dots, \sigma_{\vec{i}+\vec{v}_m}(t) \right).$$

El orden de la tupla $\mathcal{N}$ es relevante: determina la numeración de las reglas y permite definir relaciones como las reglas espejo (Sección 3.1).

2.1 Espacio finito y condiciones de frontera

Idealmente, los autómatas celulares evolucionan en espacios infinitos; sin embargo, en la práctica se establecen fronteras que restringen el espacio celular [1,3]. La condición de frontera puede ser abierta (se asigna un estado fijo a los vecinos faltantes) o periódica (los vecinos faltantes de una célula en la frontera son las células de la frontera opuesta) [3]. Un autómata infinito con una configuración periódica equivale a un autómata finito con frontera periódica [1].

En este trabajo se utiliza siempre frontera periódica. En una dimensión, el espacio es un anillo de n células, $\mathcal{L} = \mathbb{Z}_n$.

2.2 Vecindades y reglas

Para autómatas celulares unidimensionales con radio r , la vecindad contiene a todas las células a distancia menor o igual a r , incluyendo a la propia célula ($m = 2r + 1$), y la regla local se escribe como:

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \dots, \sigma_i(t), \dots, \sigma_{i+r}(t)), \quad \mathcal{R} : \mathcal{S}^{2r+1} \rightarrow \mathcal{S}.$$

Para vecindades de tamaño par se usa radio $r = 1/2$, con la convención $\mathcal{N} = (-1, 0)$: la célula depende de su vecina izquierda y de sí misma.

El autómata celular elemental (ECA) considera $k = 2$, radio $r = 1$ ($m = 3$) y la regla $\mathcal{R} : \{0, 1\}^3 \rightarrow \{0, 1\}$. Existen $k^m = 2^3 = 8$ patrones posibles en la vecindad y a cada uno se le asigna un nuevo estado, por lo que el número total de reglas es $k^{k^m} = 2^8 = 256$. Cada regla se identifica con un número decimal leyendo como número binario los estados asignados a los patrones 111, 110, $\dots$, 000.

Nótese que el número de regla solo es único cuando m y k son fijos: dos reglas que difieren en vecindad o en número de estados pueden compartir el mismo número.

2.3 Función global y grafo de transición

Sea $C = \mathcal{S}^{\mathcal{L}}$ el conjunto de configuraciones, con $N = |C| = k^{|\mathcal{L}|}$. La función global $F : C \rightarrow C$ aplica la regla local a todas las células de forma síncrona:

$$F(\sigma)_{\vec{i}} = \mathcal{R} \left(\sigma_{\vec{i}+\vec{v}_1}, \dots, \sigma_{\vec{i}+\vec{v}_m} \right), \quad \forall \vec{i} \in \mathcal{L}.$$

El grafo de transición $G_F = (C, E)$ tiene un nodo por configuración y una arista dirigida $\sigma \rightarrow F(\sigma)$. Como F es una función, cada nodo tiene grado de salida 1 (grafo funcional); por lo tanto, cada componente conexa contiene exactamente un ciclo, que contiene un atractor del cual cuelgan árboles transitorios que forman su cuenca. Para cada $x \in C$ definimos su periodo $p(x)$ como la longitud del ciclo al que llega, y su altura $h(x)$ como el mínimo número de pasos para alcanzarlo ($h(x) = 0$ si x es periódica). Las configuraciones sin predecesor se conocen como Jardín del Edén. Denotamos por $A \in 0, 1^{N \times N}$ la matriz de adyacencia de G_F .

2.4 Cociente por traslaciones

Sea T la traslación de una posición en $\mathcal{L}$ (una rotación en el anillo $\mathbb{Z}_n$). Con frontera periódica, la función global conmuta con la traslación, $F \circ T = T \circ F$, por lo que dos configuraciones que difieren por una traslación tienen trayectorias equivalentes. Esto permite construir el grafo cociente, cuyos nodos son las clases de configuraciones bajo traslación y cuyas aristas son $[x] \rightarrow [F(x)]$. Todas las configuraciones de una misma clase tienen la misma altura, y el número de nodos se reduce aproximadamente a $N/|\mathcal{L}|$. Cuando se calculen proporciones sobre el grafo cociente, cada clase se pondera por su tamaño.

3 Exact relations: Inheritance properties

Este método surge como una extensión de los criterios de equivalencia ya conocidos. Los cuales son permutación de estados y reglas espejo. La idea central es que existen relaciones adicionales entre reglas de diferentes espacios. La regla 0 es el mejor representante, ya que no importa el número de estados, el tamaño de la vecindad ni la dimensionalidad, todas las reglas 0 tienen el mismo comportamiento. Con esto surge la siguiente pregunta: ¿Cuáles son los criterios de similitud entre reglas de diferentes espacios?

Para esto, llamaremos dominio de una regla a la 3-tupla de dimensión, estados y vecindad en la que está definida. Dos reglas pertenecen al mismo dominio si comparten los tres elementos.

3.1 Reglas del mismo dominio

Para comparar reglas en el mismo dominio existe dos relaciones ampliamente utilizadas. En ECA, son capaces de reducir las 256 reglas a 88 comportamientos [2] mediante la permutación de estados y las reglas espejo.

Sean $\mathcal{A}_1$ y $\mathcal{A}_2$ dos autómatas celulares del mismo dominio, y sea $(x_1, \dots, x_m) \in S^m$ el estado de la vecindad. Decimos que $\mathcal{A}_1$ y $\mathcal{A}_2$ son equivalentes si se cumple alguna de las siguientes condiciones:

- **Permutación de estados:** existe una biyección de estados $\pi : S \rightarrow S$ tal que

$$\mathcal{R}_2(x_1, \dots, x_m) = \pi \left(\mathcal{R}_1 \left(\pi^{-1}(x_1), \dots, \pi^{-1}(x_m) \right) \right), \quad \forall (x_1, \dots, x_m) \in S^m.$$

Es decir, si cambiamos todos los estados, la estructura sigue siendo la misma. Para $S = \{0, 1\}$, la única permutación es $\pi(x) = 1 - x$, que intercambia los dos estados (los ceros por los unos y viceversa). Por ejemplo, en ECA la regla 0 es equivalente a la regla 255.

- **Reglas espejo:** sea $M : S^m \rightarrow S^m$ la reflexión de la vecindad, $M(x_1, \dots, x_m) = (x_m, \dots, x_1)$. Decimos que $\mathcal{A}_2$ es la regla espejo de $\mathcal{A}_1$ si

$$\mathcal{R}_2(x_1, \dots, x_m) = \mathcal{R}_1(M(x_1, \dots, x_m))$$

Es decir, si reflejamos la configuración inicial, la evolucionamos con $\mathcal{A}_1$ y reflejamos el resultado, obtenemos exactamente la evolución de $\mathcal{A}_2$: el diagrama espacio-tiempo de una regla es el reflejo horizontal del de la otra. Por ejemplo, en ECA la regla 110 es equivalente a la regla 124.

3.2 Reducción de estados

Sea la regla $\mathcal{A}$. La reducción de estados es posible cuando existen dos estados $a \neq b \in S$ que cumplen las siguientes condiciones:

1. El estado a no es producido por la regla: $\mathcal{R}(x_1, \dots, x_m) \neq a, \forall (x_1, \dots, x_m) \in S^m$.
2. El estado a se comporta igual que b en cualquier posición de la vecindad: $\mathcal{R}(x_1, \dots, a, \dots, x_m) = \mathcal{R}(x_1, \dots, b, \dots, x_m)$.

Entonces el autómata reducido es $\mathcal{A}' = (\mathcal{L}, S', \mathcal{N}, \mathcal{R}')$ con $S' = S \setminus \{a\}$.

La justificación es la siguiente: como el estado a nunca es producido, solo puede aparecer en la condición inicial, y como su transición coincide con la de b , ambas tienen el mismo comportamiento para toda configuración σ . Por lo tanto, a partir del primer paso la evolución de $\mathcal{A}$ es exactamente la de $\mathcal{A}'$. En el grafo de transición, las configuraciones que contienen al estado a son Jardín del Edén que se agregan a los árboles de $\mathcal{A}'$.

3.3 Reducción de vecindad

La reducción de vecindad es posible cuando el estado de un vecino no afecta a la transición, es decir, existe un índice j tal que:

$$\mathcal{R}(x_1, \dots, x_j, \dots, x_m) = \mathcal{R}(x_1, \dots, x'_j, \dots, x_m), \quad \forall (x_1, \dots, x_m) \in S^m, \forall x'_j \in S.$$

Entonces el autómata reducido es $\mathcal{A}' = (\mathcal{L}, S, \mathcal{N}', \mathcal{R}')$ con $\mathcal{N}' = \mathcal{N} \setminus \{\vec{v}_j\}$.

Si el vecino eliminado no forma parte de los extremos, entonces se realiza un ajuste en la vecindad que la vuelve conigua de $m - 1$ células.

3.4 Reducciones adicionales

De forma análoga se puede definir la reducción de dimensionalidad, cuando la regla no depende de uno de los ejes del espacio (por ejemplo, una regla en $\mathbb{Z}^2$ que imita a un

ECA sin depender del eje vertical). Esta reducción no se considera en este trabajo, pues el objeto de estudio se limita a una dimensión.

Otra reducción posible es la de distancia vecinal: considerar a los vecinos por dirección y no por distancia, de modo que los vectores de desplazamiento se compriman a una vecindad contigua. Por ejemplo, una regla con vecindad $(-1, 1)$ se compara con la misma regla con vecindad $(-1, 0)$. Esta compresión solo está definida si no hay dos vecinos en la misma dirección (por ejemplo, no puede haber un vecino en -3 y otro en -1). Una hipótesis es que el comportamiento de ambas reglas es el mismo bajo una distorsión del espacio (Fig. 1).

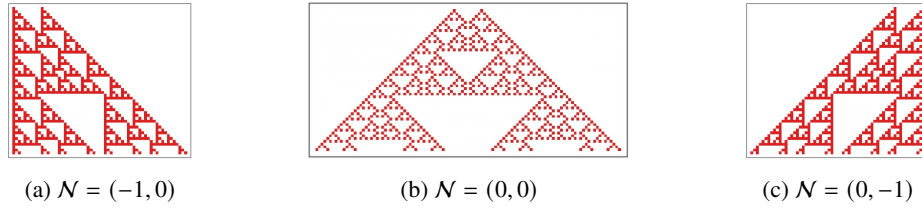


Fig. 1: Evolución de la regla ECA 6 con la misma condición inicial pero con diferentes vectores de desplazamiento.

3.5 Clases de equivalencia

Con estas cuatro relaciones (permutación de estados, espejo, reducción de estados y reducción de vecindad) es posible relacionar reglas de dominios distintos. Aplicándolas de forma iterativa se construye un grafo de relaciones, cuyas componentes conexas son las clases de equivalencia. Aunque esta clasificación sigue siendo binaria (dos reglas son o no son equivalentes), funciona como referencia para validar los métodos siguientes: si dos reglas son equivalentes, su índice de similitud debería ser uno.

Mediante reducción de vecindad se pueden obtener una reducción de los comportamientos únicos. En ECA se reduce de 88 a 81 clases de equivalencia. Algunas de estas clases se unen porque sus reglas se reducen a la misma regla de un dominio menor. Por ejemplo, la regla 10 $(\neg l \wedge r)$ y la regla 12 $(\neg l \wedge c)$ se reducen a la regla 2 de dos vecinos $(\neg a \wedge b)$; la primera al eliminar la célula central y la segunda al eliminar el vecino derecho.

4 Structural comparison of transition graphs

Las relaciones de la Sección 3 se definen sobre la regla local. Una alternativa es comparar directamente la dinámica global: para un espacio de n células existen $N = |S|^n$ configuraciones, y al evolucionar todas ellas un paso se obtiene el grafo de transición G_F de la regla. Decimos que dos reglas tienen el mismo comportamiento en un espacio de n células si sus grafos de transición tienen la misma topología, es decir, si son isomorfos.

Estos métodos tienen dos restricciones principales:

1. La comparación sigue siendo binaria.
2. El resultado depende del tamaño del espacio n : dos reglas pueden tener grafos isomorfos para un valor de n y no para otro, y el número de comportamientos distinguibles cambia con n .
3. La comparación unicamente es entre reglas del mismo dominio.

4.1 Isomorfismo y forma canónica

La comparación isomorfica identifica con total exactitud las reglas por permutación y las reglas espejos, corroborando que la topología de esas reglas es identica.

Este proceso resulta computacionalmente costoso. Una alternativa es la representación del grafo evolutivo en su forma canónica: una codificación que no depende de las etiquetas de los nodos, de modo que dos grafos son isomorfos si y solo si sus formas canónicas son iguales. Así, cada grafo se codifica una sola vez y comparar reglas se reduce a comparar códigos. Aprovechando que G_F es un grafo funcional (cada componente es un ciclo con árboles colgando), la forma canónica se construye de la siguiente manera:

1. Se agregan a una cola todos los nodos sin predecesor (Jardín del Edén).
2. Se extrae un nodo de la cola y se le asigna como código la lista ordenada de los códigos de sus predecesores (vacía para los Jardín del Edén). Este código se agrega a la lista de predecesores de su sucesor.
3. Si el sucesor ya no tiene predecesores sin procesar, se agrega a la cola. Se repite hasta vaciar la cola.
4. Los nodos que nunca entraron a la cola son exactamente los nodos de los ciclos. Cada ciclo se codifica como la secuencia de los códigos de sus nodos (cada uno con el árbol que cuelga de él), tomando la rotación lexicográficamente menor para que no dependa del nodo inicial.
5. La forma canónica es la lista ordenada de los códigos de todos los ciclos.

Dos reglas tienen la misma forma canónica si y solo si sus grafos de transición son isomorfos.

Al agrupar las reglas por forma canónica se obtienen las clases de comportamiento en un espacio de n células. El número de clases crece con n , pero no indefinidamente: en ECA, con espacios de 4 células ya se distinguen casi todos los comportamientos conocidos. Esto plantea la pregunta de cuál es el tamaño mínimo del espacio que conserva la información necesaria para distinguir las reglas.

4.2 Forma canónica en GPU

Calcular la forma canónica para las reglas ECA solo era viable hasta espacios de alrededor de 10 células. Para explorar espacios más grandes implementamos el mismo proceso en la GPU (CUDA desde Mathematica): los sucesores, el grado de entrada y la poda desde los Jardín del Edén se calculan en paralelo con un hilo por configuración, y el código de cada árbol se representa con un hash de 64 bits del multiconjunto de los códigos de sus hijos, acumulado con sumas atómicas. Los ciclos se compactan en la

GPU y su rotación mínima se obtiene en CPU. Para $n = 1, \dots, 10$ y las 256 reglas, la partición obtenida es idéntica a la de la forma canónica exacta. El límite es la memoria de la tarjeta (16 GB): unos 20 bytes por configuración permiten llegar a $n = 29$ (2^{29} configuraciones). Como las reglas relacionadas por permutación de estados o espejo tienen grafos isomorfos, basta evolucionar un representante de cada una de las 88 clases.

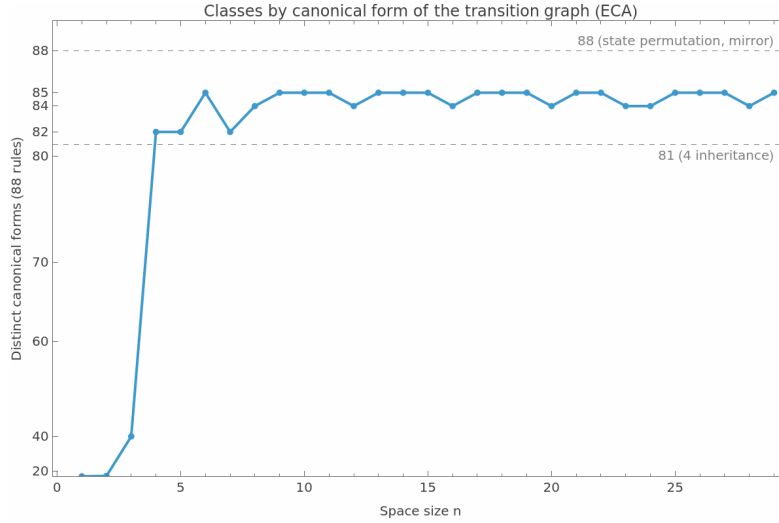


Fig. 2: Número de formas canónicas distintas entre las 88 reglas representantes para $n = 1, \dots, 29$ (eje vertical en escala cúbica). Las líneas punteadas marcan las 88 clases por permutación de estados y espejo y las 81 clases con las cuatro relaciones de herencia.

Table 1: Número de formas canónicas distintas entre las 88 reglas representantes en un espacio periódico de n células.

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Clases	3	9	40	82	82	85	82	84	85	85	85	84	85	85	85
n	16	17	18	19	20	21	22	23	24	25	26	27	28	29	
Clases	84	85	85	85	84	85	85	84	84	85	85	85	84	85	

El número de clases se estabiliza en 84 o 85 a partir de $n = 8$ (Figura 2 y Tabla 1), pero ningún tamaño alcanza las 88: en cada n hay pares de reglas sin relación de permutación de estados ni espejo cuyos grafos de transición son isomorfos. Con $n \leq 3$ se agrupan muchas reglas (grupos de 40, 24 y 24 reglas con $n = 1$). La Tabla 2 muestra las reglas agrupadas a partir de $n = 4$.

A partir de $n = 8$ los pares agrupados dependen de la paridad del espacio: con n par siempre se agrupan $\{15, 170\}$, $\{23, 178\}$ y $\{77, 232\}$, y si n es múltiplo de 4 también

Table 2: Reglas representantes con la misma forma canónica en un espacio de n células.

n	Clases	Reglas agrupadas
4	82	{15,170}, {23,178}, {77,232}, {105,150}, {25,45}, {28,78}
5	82	{10,34}, {24,46}, {136,160}, {28,122}, {36,72}, {44,78}
6	85	{15,170}, {23,178}, {77,232}
7	82	{10,34}, {24,46}, {136,160}, {15,105}, {60,90}, {150,170}
8, 12, 16, 20, 24, 28	84	{15,170}, {23,178}, {77,232}, {105,150}
10, 14, 18, 22, 26	85	{15,170}, {23,178}, {77,232}
9, 11, 13, 15, 17, 19, 21, 25, 27, 29	85	{10,34}, {24,46}, {136,160}
23	84	{10,34}, {24,46}, {136,160}, {60,90}

{105,150}; con n impar siempre se agrupan {10,34}, {24,46} y {136,160}. La única excepción es {60,90}, que se agrupan en $n = 7$ y $n = 23$.

Como los pares agrupados en espacios pares no se agrupan en espacios impares y viceversa, combinar dos tamaños recupera los 88 comportamientos: dos reglas se consideran iguales solo si tienen la misma forma canónica en ambos espacios. La Tabla 3 muestra el número de clases al combinar dos tamaños. Ningún tamaño por sí solo, ni ninguna combinación de dos tamaños pares o de dos impares con $n \leq 29$, alcanza las 88 clases; siempre se necesita un espacio par y uno impar. Sin embargo, no toda combinación par-impar funciona ({1, 4} da 86, {2, 3} da 75 y {1, 9} da 85). La combinación mínima es $n = 3$ y $n = 4$: con únicamente estos dos espacios, es decir, con solo $2^3 + 2^4 = 24$ configuraciones, se distinguen los 88 comportamientos de las reglas ECA. Le siguen {2, 5}, {4, 5}, {1, 6}, {3, 6} y {5, 6}, y a partir de $n = 5$ cualquier combinación de un tamaño par y uno impar da 88 clases.

Table 3: Número de clases al combinar la forma canónica en dos tamaños de espacio n_1 y n_2 (la diagonal corresponde a un solo tamaño). Las combinaciones que distinguen los 88 comportamientos están en negritas.

$n_1 \backslash n_2$	1	2	3	4	5	6	7	8	9	10
1	3	10	40	86	82	88	82	88	85	88
2	10	9	75	83	88	85	88	85	88	85
3	40	75	40	88	85	88	85	88	85	88
4	86	83	88	82	88	85	88	84	88	85
5	82	88	85	88	82	88	85	88	85	88
6	88	85	88	85	88	85	88	85	88	85
7	82	88	85	88	85	88	82	88	85	88
8	88	85	88	84	88	85	88	84	88	85
9	85	88	85	88	85	88	85	88	85	88
10	88	85	88	85	88	85	88	85	88	85

5 Spectral methods

Los métodos de la Sección 4 solo responden si dos reglas son iguales o no. Para obtener una medida gradual, busquemos representar cada regla con un vector numérico derivado de su grafo de transición, de modo que la distancia entre vectores funcione como índice de similitud.

5.1 Eigenvalores

En este método se calculan los eigenvalores de la matriz de adyacencia A del grafo de transición, se ordenan y se usan como vector de características de la regla; dos reglas se comparan mediante la distancia coseno entre sus vectores. Si dos grafos son isomorfos, sus matrices de adyacencia son similares por una permutación y producen los mismos eigenvalores, por lo que la distancia es cero. En caso contrario, la distancia mide qué tan distintos son sus espectros. Con este método se obtienen 74 comportamientos distintos en ECA con un espacio de 18 células.

Esto explica las dos limitaciones principales del método.

Ceguera a los transitorios. Como los árboles transitorios solo aportan eigenvalores nulos, el espectro no distingue cómo se distribuyen las configuraciones en las cuencas ni cuántos pasos tardan en llegar al atractor. Por ejemplo, las reglas 0 y 8 de ECA no están relacionadas por ninguna de las relaciones de la Sección 3, ni siquiera después de reducir vecinos, y sus grafos de transición no son isomorfos. Sin embargo, en ambas el único atractor es el punto fijo formado por puros ceros: en la regla 0 todas las configuraciones llegan a él en un paso, mientras que en la regla 8 algunas tardan dos. Ambas tienen el mismo espectro (un eigenvalor 1 y $N - 1$ eigenvalores 0), por lo que el método las considera idénticas. Este caso no es excepcional, lo que explica que el número de comportamientos distintos nunca alcance al obtenido con la forma canónica.

Dependencia del dominio. El número de eigenvalores es igual al número de nodos $N = |S|^n$, por lo que solo se pueden comparar reglas con el mismo número de estados y el mismo tamaño de espacio.

5.2 Momentos espectrales

Para resolver la dependencia del dominio proponemos usar los momentos espectrales. En lugar de calcular el espectro completo, para cada paso $k = 1, \dots, K$ se mide la proporción de configuraciones que regresan a sí mismas tras k pasos:

$$m_k = \frac{|\{x \in C : F^k(x) = x\}|}{N}$$

La ventaja es que la longitud del vector es K y no depende del tamaño del grafo, así que ahora es posible comparar reglas con distinto número de estados, vecindades o tamaños de espacio. Las reglas relacionadas por reducción de estados o de vecindad (Sección 3) sirven como referencia para validar estas comparaciones entre dominios. La

desventaja es que K debe elegirse de modo que los ciclos relevantes de ambos grafos queden dentro de la ventana. Los vectores se comparan mediante la distancia coseno, que es nuestro índice de similitud.

En ECA, con el grafo cociente por rotaciones de 14 células y $K = 10$, se obtienen 63 vectores distintos para las 256 reglas.

Sin embargo, como m_k solo depende de los eigenvalores, este método hereda la ceguera a los transitorios: las reglas 0 y 8 tienen $m_k = 1/N$ para todo k .

5.3 Momentos espectrales con masa de atractor

Para distinguir reglas como la 0 y la 8 es necesario medir, además de los atractores, la fuerza con la que atraen a las demás configuraciones: cuántas configuraciones llegan a cada atractor y en cuántos pasos. La idea es que cada ciclo se siga activando cada p pasos, como en los momentos espectrales, pero que en cada activación se sumen también las configuraciones que ya llegaron a él.

Usando la altura $h(x)$ y el periodo $p(x)$ de cada configuración, definimos

$$\tilde{m}_k = \frac{|\{x \in C : p(x) \mid k \text{ y } h(x) \leq k - p(x)\}|}{N}, \quad k = 1, \dots, K.$$

Es decir, una configuración cuenta en el paso k si su atractor se activa en ese paso y ella ya había llegado a él antes de la activación anterior. Así, los nodos del ciclo ($h = 0$) cuentan desde la primera activación ($k = p$), los nodos con $1 \leq h \leq p$ desde la segunda ($k = 2p$), los nodos con $p < h \leq 2p$ desde la tercera, y así sucesivamente.

Como toda configuración periódica cumple la condición cuando $p \mid k$, el vector se descompone como

$$\tilde{m}_k = m_k + t_k,$$

donde m_k es el momento espectral y t_k es la proporción de configuraciones transitorias que cuentan en el paso k . Por lo tanto, el nuevo vector extiende a los momentos espectrales con la información de las cuencas, y conserva la ventaja de tener longitud K independiente del dominio.

Por ejemplo, con 5 células ($N = 32$), la regla 0 envía todas las configuraciones a cero en un paso ($h \leq 1$), mientras que en la regla 8 cada bloque de unos deja un 1 aislado que desaparece en el siguiente paso ($h \leq 2$); de las 32 configuraciones, 12 llegan a cero en a lo más un paso. Sus vectores son

$$\tilde{m}^{(0)} = \frac{1}{32} (1, 32, 32, 32, \dots), \quad \tilde{m}^{(8)} = \frac{1}{32} (1, 12, 32, 32, \dots),$$

que ya no son iguales, aunque sus momentos espectrales sí lo son.

Un ejemplo con ciclos más largos es la regla 30 con 5 células (Fig. ??). Su grafo tiene dos componentes: el punto fijo 0, cuya cuenca es $\{0, 31\}$, y un ciclo de longitud 5 del que cuelgan cinco cadenas de cinco configuraciones cada una, con una cuenca de 30 configuraciones. En $k = 1$ solo cuenta el punto fijo, y desde $k = 2$ también la configuración 31. En $k = 5$ se activa por primera vez el ciclo y cuentan solo sus 5 nodos; en $k = 10$ cuentan además las 25 configuraciones transitorias, que llegaron al ciclo en a lo más 5 pasos:

$$\tilde{m}^{(30)} = \frac{1}{32} (1, 2, 2, 2, 7, 2, 2, 2, 2, 32, 2, 2, 2, 2, 32, \dots).$$

Si alguna configuración tardara 7 pasos en llegar al ciclo de longitud 5, contaría hasta $k = 15$, la tercera activación.

Este método tiene algunas limitaciones. La resolución con la que se mide la altura depende del periodo, pues las configuraciones se agrupan en intervalos de p pasos: con periodos largos, una configuración con $h = 1$ y otra con $h = p$ cuentan en el mismo paso. Además, un ciclo con $p > K$ nunca se activa. Finalmente, en la reducción de estados las configuraciones que contienen el estado eliminado son Jardín del Edén adicionales que llegan un paso más tarde y cambian la normalización N , por lo que es necesario verificar cómo se comportan los pares relacionados por esta reducción.

Acknowledgments. TODO: acknowledgments.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Bhattacharjee, K.: Cellular Automata: Reversibility, Semi-Reversibility and Randomness. Ph.D. thesis, Indian Institute of Engineering Science and Technology, Shibpur (2019), <https://arxiv.org/abs/1911.03609>, last accessed 2025/08/23
2. Martinez, G.J.: A note on elementary cellular automata classification. *Journal of Cellular Automata* **8** (2013)
3. Ureña Ponce, O.: Diseño de un procesador de autómatas celulares para la simulación de sistemas complejos. Master's thesis, Universidad de Guadalajara, Guadalajara, México (2015), <https://riudg.udg.mx/bitstream/20.500.12104/79915/1/MCUCEI01068FT.pdf>
4. Sancho Caparrini, F.: Autómatas celulares. Universidad de Sevilla (2016), https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html, last accessed 2025/09/24
5. Weisstein, E.W.: Cellular automaton. *MathWorld – A Wolfram Resource* (2002), <https://mathworld.wolfram.com/CellularAutomaton.html>, last accessed 2025/07/23
6. Wolfram, S.: Cellular automata as simple self-organizing systems. Technical Report CALT-68-938, California Institute of Technology (1982), <https://content.wolfram.com/sw-publications/2020/07/cellular-automata-self-organizing-systems.pdf>